Debate-Driven Multi-Agent Security Decision for Distributed Web Threat Triage: Design and Evaluation of AetherGuard

Anonymous Author[1*]

[1*]Anonymous Institution, Anonymous Organization, Anonymous City, Anonymous Country.

Corresponding author(s). E-mail(s): anonymous@example.com;

Abstract

Distributed web defense increasingly mixes centralized policy control with local enforcement, yet most AI-assisted security pipelines still rely on a single model invocation that produces brittle labels and weak operational explanations. This paper presents AetherGuard, a debate-driven multi-agent triage architecture for ambiguous web threats in distributed environments. The method assigns three explicit roles to security agents: Socrates challenges a request from an adversarial perspective, Plato evaluates defensive handling and policy impact, and Aristotle aggregates evidence into a final verdict distribution and case file. Debate is triggered only when label conflict, confidence spread, or distribution divergence indicates disagreement, so the mechanism targets hard cases instead of adding unconditional latency. The implementation couples this triage loop with authenticated WebSocket gateways, signed command dispatch, audit-chain retention, and local/cloud/hybrid routing modes. Public evidence is drawn from a repository-traceable OWASP-derived corpus of 32 balanced samples spanning SQL injection, cross-site scripting, path traversal, and command injection. On this corpus, the debate-enabled configuration improves accuracy from 87.5% to 100% and removes all four observed false positives while keeping mean orchestration latency at 0.5 ms. A second evidence layer is prepared as a reporting protocol for private labeled alerts. The resulting draft positions AetherGuard as a system-and-method contribution centered on auditable multi-agent security judgment rather than platform breadth claims.

Keywords: multi-agent security, web threat triage, debate-based reasoning, explainable security operations, distributed defense

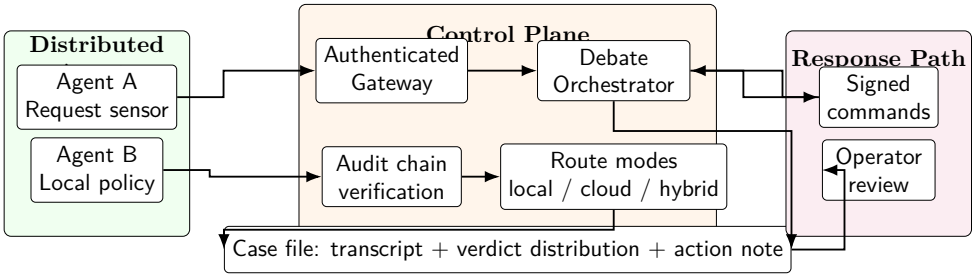


Fig. 1 Overview of the paper's system-and-method scope.

1 Introduction

Distributed defense platforms increasingly split security execution across many servers while retaining centralized policy control. This arrangement improves local response speed, but it also exposes a decision bottleneck: once an observed request becomes ambiguous, a single classifier or a single large language model must determine whether to allow, monitor, or block it. Prior intrusion detection systems and modern AI-assisted detectors both suffer from this bottleneck in different forms. Classical systems such as Snort and Bro provide deterministic responses with limited semantic flexibility [2, 3], while learning-based detectors improve coverage but still concentrate judgment into one predictive boundary [4–6]. In operational web defense, this often translates into an undesirable trade-off: conservative thresholds raise false positives, whereas aggressive suppression of false alarms can undercut recall.

This paper positions AetherGuard around a narrower question than the broader platform literature: *can role-specialized agent debate improve the triage of ambiguous web threats without losing auditability or distributed deployability?* The answer matters because security operators do not merely need labels. They also need a justification trail, a clear handling recommendation, and a record that can be reviewed after an escalation or a false alarm.

The repository already contains the ingredients for such a study. The backend exposes an authenticated agent gateway, HMAC-signed command dispatch, audit retention, configurable local/cloud/hybrid routing, and a multi-agent orchestration stack. More importantly, the current workspace contains a traceable public evaluation on an OWASP-derived benchmark slice and an explicit validated-capability snapshot. These artifacts support a paper that is more conservative, but also more defensible, than a platform-wide whitepaper.

Figure 1 summarizes the target argument of the paper. Rather than claiming that every subsystem of AetherGuard is independently novel, the draft centers one primary mechanism: debate-triggered security judgment over ambiguous web requests. The rest of the platform is presented only insofar as it supports that mechanism in a distributed setting.

The paper makes three contributions.

1. It defines a **role-specialized debate mechanism** for web threat triage in which Socrates performs adversarial challenge, Plato evaluates defensive handling, and Aristotle aggregates and arbitrates the final decision.
2. It introduces an **auditable triage pipeline** that converts role outputs into a debate transcript, a final verdict distribution, and a case file suitable for post-incident review.
3. It demonstrates these ideas in **the AetherGuard implementation** using repository-traceable public evidence: a 32-sample balanced OWASP-derived corpus and a validated smoke benchmark snapshot for system overhead.

The remainder of the paper is organized as follows. Section 2 reviews the three literature lines that matter for this narrowed thesis: web attack detection, AI-assisted security analysis, and multi-agent debate. Section 3 formalizes the debate-driven triage method. Section 4 maps that method onto the relevant AetherGuard implementation surfaces. Section 5 reports the public evidence layer and defines the protocol for a second, private labeled-alert layer. Section 6 discusses limitations and concludes.

2 Related Work

2.1 Web Attack Detection and Triage

Intrusion detection has evolved from rule-centric monitoring to increasingly adaptive learning-based pipelines. Denning's early anomaly model framed detection as a deviation-identification problem [1], while Snort and Bro demonstrated the durability of interpretable packet and log inspection in production networks [2, 3]. More recent systems use deep models to classify network or application behavior with better coverage against evolving threats [4, 6]. Yet the dominant output abstraction has not changed much: a request or flow is eventually mapped to a single label with a score.

For web threat triage, this is a practical weakness rather than a purely statistical one. Operators need to know not just whether a request appears malicious, but why it should be blocked now, whether the evidence is conflicting, and whether the same request family would benefit from a policy update. The current paper addresses this operational gap by treating a request verdict as the output of explicit role conflict resolution rather than of one opaque model invocation.

2.2 AI-Assisted Security Analysis and Explainability

The explainable IDS literature shows growing dissatisfaction with bare classification scores [5]. Feature-attribution methods can help analysts understand model focus, but they often remain post-hoc and model-centric. In contrast, security operations typically require procedural traceability: what evidence was considered, which disagreement mattered, and why a final handling decision was selected. Surveys and recent applied work argue that security teams need explanations tied to incident workflow rather than only to feature importance [5, 17].

This distinction is central to our positioning. AetherGuard does not attempt to introduce another post-hoc explanation layer. Instead, the explanation object is native

to the triage workflow itself: debate transcript, verdict distribution, and case file. This is a narrower and more defensible claim than broad "explainable AI" language, and it aligns better with the artifact surfaces already present in the codebase.

2.3 Multi-Agent Debate and Routing

General-purpose LLM agent frameworks show that role assignment and inter-agent communication can improve complex task performance [7–9]. Debate-based reasoning goes further by externalizing disagreement instead of hiding it inside one model pass [10]. This direction is especially relevant after evidence that LLMs do not reliably self-correct reasoning without structured external feedback [11, 12]. Our triage method adopts that insight in a security-specific form: debate is only triggered when disagreement indicators justify it, and each role is grounded in a recognizable security posture rather than a generic team structure.

Routing and privacy constraints form the second ingredient. Cost-aware routing systems such as FrugalGPT and RouteLLM show that multi-model inference can be treated as a decision problem [14, 15]. Security deployments add stricter requirements around local processing, content redaction, and fallback behavior [13]. The AetherGuard implementation uses these routing ideas as system support, but not as the main contribution. In this paper, routing exists to make debate-driven triage deployable across privacy modes and runtime availability conditions.

2.4 Positioning Against Collaborative Security

Collaborative intrusion detection research traditionally focuses on intelligence sharing and distributed trust [16]. That line is important background because AetherGuard is also distributed. However, our target problem differs in scope. The present paper is not about cross-organization alert federation, consensus trust, or multi-party data fusion. It is about the internal decision quality of a distributed security platform when one web request produces conflicting signals.

This leads to a compact positioning statement: the contribution is *not* a broader multi-agent platform, a broader distributed IDS, or a broader explainability framework. It is a debate-centered decision method, implemented inside AetherGuard, and evaluated first on repository-traceable public evidence.

3 Method

3.1 Task Definition

The target task is request-level web threat triage in a distributed defense system. Given an observed request r, the system must emit a handling decision $d \in \{\texttt{allow}, \texttt{monitor}, \texttt{challenge}, \texttt{block}\}$, together with a confidence-bearing rationale that can be reviewed later. The core object is not a single label, but a structured decision package:

$$\mathcal{Y}(r) = \langle \mathbf{p}, d, \tau, \kappa \rangle$$

where $\mathbf{p}$ is the final verdict distribution over `safe`, `suspicious`, and `malicious`; τ is the debate transcript; and κ is the resulting case file.

Table 1 Core symbols and operational objects.

Symbol	Meaning
r	observed web request with path, headers, body, and source metadata
$\mathcal{A}$	role set {Socrates, Plato, Aristotle}
$\mathbf{p}_i$	role-specific verdict distribution over {safe, suspicious, malicious}
c_i	role-specific confidence score
$\delta(r)$	disagreement state computed from role outputs
τ	debate transcript generated only when disagreement is material
κ	case file containing verdict, rationale, evidence summary, and follow-up action
m	routing mode from {cloud_only, local_preferred, hybrid_verify}

Table 1 defines the main symbols and interfaces used in the remainder of the paper. The formulation matches the repository implementation, in which each role emits a verdict distribution and the orchestration layer computes disagreement statistics before deciding whether debate is necessary.

3.2 Three-Role Debate Pipeline

The method uses three fixed roles rather than a larger open-ended agent society. This is deliberate. In security operations, excess agent diversity can add verbosity without improving actionability. The three roles used here correspond to common operational viewpoints:

- **Socrates**: adversarial challenge. This role assumes the request may be a bypass attempt or an underestimated attack mutation.
- **Plato**: defensive handling. This role reasons about response feasibility, policy consequences, and whether the evidence justifies intervention.
- **Aristotle**: synthesis and arbitration. This role aggregates the other views and is responsible for final verdict formation.

Each role emits an initial tuple $\langle \mathbf{p}_i, c_i, e_i \rangle$ containing a verdict distribution, a confidence score, and supporting evidence fragments. The first-stage result is therefore a set of parallel opinions rather than a single score.

3.3 Debate Triggering

Debate is not unconditional. It is only invoked when disagreement indicators suggest that the request is ambiguous enough to deserve explicit role conflict resolution. The trigger state is computed from three signals already present in the implementation:

1. **label conflict**: whether the dominant labels of the role distributions differ;
2. **distribution divergence**: the average pairwise Jensen–Shannon divergence across $\mathbf{p}_i$;
3. **confidence range**: $\max_i c_i - \min_i c_i$.

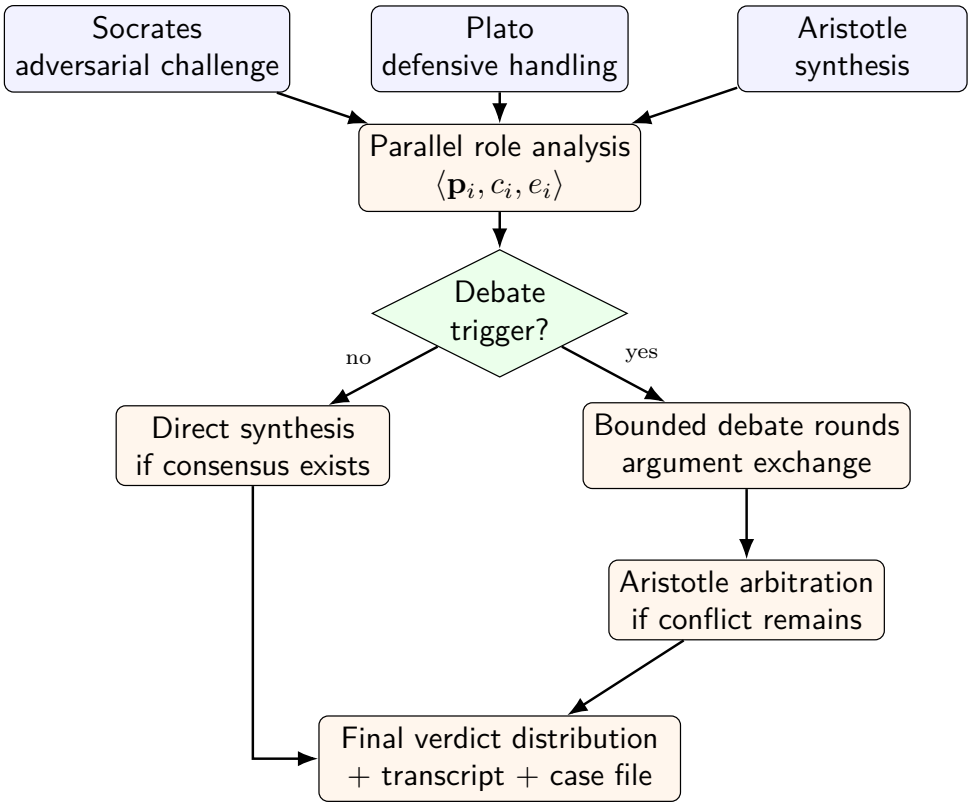


Fig. 2 Decision flow of the three-role debate pipeline.

Let τ_d and τ_c denote divergence and confidence thresholds. Debate is triggered when

$$\begin{aligned} \delta(r) = \mathbb{I}[&\text{label conflict}] \\ &\vee \mathbb{I}[\mathrm{JSD}_{avg}(\mathbf{p}_1, \mathbf{p}_2, \mathbf{p}_3) > \tau_d] \\ &\vee \mathbb{I}[\max_i c_i - \min_i c_i > \tau_c]. \end{aligned} \tag{1}$$

This matches the repository scoring logic and intentionally focuses debate on hard cases instead of easy consensus cases.

3.4 Arbitration and Auditable Outputs

When debate is triggered, the three roles enter bounded rounds of argument exchange. The implementation uses a small maximum round budget and then falls back to Aristotle arbitration. This keeps the pipeline operationally predictable. The final verdict is produced by blending role distributions into an aggregate distribution $\bar{\mathbf{p}}$, from which the dominant label and confidence are extracted. Unlike a plain majority vote, the

Algorithm 1 Debate-driven triage for request r

Require: observed request r, route mode m, role set $\mathcal{A}$

for all $a_i \in \mathcal{A}$ **do**
    compute initial role output $\langle \mathbf{p}_i, c_i, e_i \rangle$
end for
compute disagreement state $\delta(r)$
if $\delta(r)$ is true **then**
    execute bounded debate rounds
    let Aristotle arbitrate unresolved conflict
end if
blend role distributions into $\bar{\mathbf{p}}$
derive final handling decision d
synthesize transcript τ and case file κ
return $\langle \bar{\mathbf{p}}, d, \tau, \kappa \rangle$

aggregate preserves uncertainty shape and can therefore distinguish between strong malicious consensus and weak suspicious consensus.

The crucial design choice is that the method outputs more than d. It emits three auditable products:

- the **debate transcript**, which records why disagreement existed;
- the **final verdict distribution**, which records the shape of remaining uncertainty;
- the **case file**, which records the rationale and recommended follow-up action.

These artifacts operationalize "explainability" as traceable decision workflow rather than as feature attribution.

3.5 Algorithm Sketch

Algorithm 1 highlights the paper's main methodological claim: explicit disagreement handling is the mechanism under study. The distributed platform, routing layer, and gateway security measures are implementation support that make this mechanism usable in practice.

4 System Realization in AetherGuard

4.1 Relevant Runtime Surfaces

The full repository contains many subsystems, but only a subset is required to support the thesis of this paper. The method described in Section III is realized through four concrete surfaces.

First, the **agent gateway** accepts authenticated WebSocket traffic from agents and dashboard clients, applies socket- and user-level rate limits, verifies tenant context, and records audit events. This is the entry point through which distributed evidence reaches the control plane. Second, **command signing** ensures that response actions sent back to an agent are integrity-protected. Third, the **platform configuration**

surface exposes routing and privacy controls used by the triage layer. Fourth, the **case-file path** persists the outcome of triage for later inspection.

The resulting system scope is narrower than the overall product scope. Hardware deployment tooling, dashboard visual polish, and general admin functionality are part of the repository, but they are peripheral to the present claim and are therefore not treated as primary research contributions.

4.2 Distributed Path From Request to Action

The control path of interest proceeds as follows:

1. a distributed agent or gateway-observed request is normalized into a traffic object;
2. the three-role triage pipeline produces initial role outputs;
3. disagreement metrics decide whether bounded debate is required;
4. the final output is synthesized into a verdict distribution and a case file;
5. if action is required, the control plane dispatches a signed command back to the relevant agent.

This path is intentionally auditable at each transition. The gateway contributes authentication and rate limiting, the command path contributes integrity, and the audit layer contributes retention and verification.

4.3 Routing and Privacy Modes

The repository currently exposes route modes including `cloud_only`, `local_preferred`, and `hybrid_verify`. These modes are not presented here as an independent contribution; they serve as deployment support for debate-driven triage. In particular, they allow the system to distinguish between operational latency mode and privacy-sensitive mode. The current validated workspace snapshot reports `cloud_only` as the active routing mode and `privacyMode=off`; this is important because the latency values reported in Section V are therefore orchestration measurements under the present configuration rather than full local-model inference measurements.

4.4 Why the System Claim Stays Conservative

The repository also includes benchmark harnesses, deployment bundles, device workflows, and whitepaper-style claims. This draft deliberately avoids turning those into broad system-performance promises. The validated workspace snapshot is mixed: it confirms the presence of benchmark infrastructure, audit-chain verification, notification outbox behavior, and device-gateway compatibility, but it does not validate the strongest historical throughput or deployment-scale statements. Therefore, the manuscript uses those artifacts only to support a limited statement: the triage method has been implemented inside a real distributed security platform with authenticated control and measurable overhead, not that every historical platform-scale claim has been reproduced in the current environment.

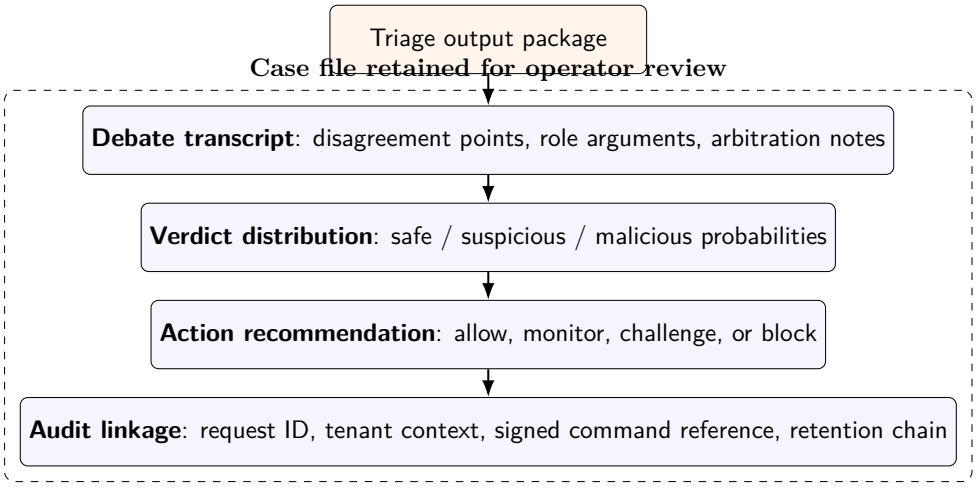


Fig. 3 Auditable outputs emphasized in this paper.

4.5 Implementation Boundary for This Draft

The system realization claim can therefore be stated precisely. AetherGuard provides:

- a concrete three-role triage implementation with disagreement-triggered debate;
- authenticated and rate-limited WebSocket control paths;
- HMAC-backed command dispatch and auditable event logging;
- deployable route modes for local/cloud/hybrid inference decisions.

This boundary is sufficient for a JCST-style system-and-method paper focused on multi-agent security judgment, and it avoids the broader platform sprawl that would otherwise dilute the paper.

5 Evaluation

5.1 Evidence Layers and Reporting Policy

The evaluation is intentionally split into two layers.

Public layer. This layer is already available in the repository and is therefore used as the current evidence base for this draft. It consists of a 32-sample balanced corpus derived from OWASP Benchmark v1.2 [18], covering four request families: SQL injection, cross-site scripting, path traversal, and command injection. The repository evaluation compares three configurations: `single_agent`, `multi_agent_no_debate`, and `multi_agent_with_debate`.

Private layer. This layer is reserved for future labeled alert data. The present manuscript specifies how such a layer should be reported, but does not fabricate its results. If timestamps are available, splitting should follow early 70%, middle 10%

Table 2 Public baseline comparison on the repository OWASP-derived 32-sample corpus.

Configuration	Accuracy	Precision	Recall	F1	False positives	False negatives
`single_agent`	0.8750	0.8000	1.0000	0.8889	4	0
`multi_agent_no_debate`	0.8750	0.8000	1.0000	0.8889	4	0
`multi_agent_with_debate`	1.0000	1.0000	1.0000	1.0000	0	0

for threshold calibration, and late 20% for final reporting. Otherwise, a fixed-seed stratified 80/20 split should be used.

5.2 Metrics

The public layer reports the metrics that directly test the thesis of the paper:

- accuracy, precision, recall, and F1;
- false positive count and false negative count;
- debate trigger ratio and average debate rounds;
- end-to-end orchestration latency.

The emphasis is deliberate. Since the paper claims that debate helps on ambiguous cases, false positives and debate trigger behavior are treated as first-class outputs rather than footnotes.

5.3 Public Baseline Comparison

Table 2 reports the main public comparison. The result is compact but important. The debate-enabled setting eliminates the four false positives that remain in both non-debate baselines, improving accuracy from 87.5% to 100% on the current balanced slice. The repository summary further notes that these four false positives all occur in benign command-injection-like requests, which is exactly the kind of ambiguity this method is meant to resolve.

In this comparison, `single_agent` uses the repository's `cloud_only` route, while `multi_agent_no_debate` preserves the role decomposition but disables explicit disagreement handling. The paper's main claim should not be overstated beyond this result. The corpus is small, balanced, and derived from a benchmark suite rather than from production traffic. Still, the pattern is coherent with the method: debate is most useful when a superficially suspicious request is actually benign.

5.4 Ablation and Overhead

Table 3 combines the evidence currently available for ablation-style interpretation. The first part isolates the effect of debate. The second part reports route-mode latency observations from the repository summary. These values are orchestration measurements and should not be interpreted as full model inference latency. The third part records the current system-overhead snapshot from the validated workspace report.

Across the public corpus, debate is triggered on 12 out of 32 requests, or 37.5%, with an average of 0.75 debate rounds per sample. This is consistent with the intended

Table 3 Repository-traceable ablation and overhead evidence.

Evidence slice	Setting	Observed metric	Interpretation
Debate ablation	no debate	accuracy 0.8750; 4 false positives	retains all four benign command-injection-like false positives
Debate ablation	with debate	accuracy 1.0000; mean orchestration latency 0.5000 ms	removes all four false positives on the public slice
Route mode	`cloud_only`	mean orchestration latency 0.8438 ms	highest observed orchestration overhead in the repository summary
Route mode	`local_preferred`	mean orchestration latency 0.7500 ms	lower orchestration overhead than `cloud_only`
Route mode	`hybrid_verify`	mean orchestration latency 0.7813 ms; debate latency 0.0938 ms	lowest reported debate latency among the route-mode entries
Workspace smoke benchmark	fastpath snapshot	1329.79 QPS; 21 ms p95 latency	validated smoke profile only, not a full production-scale claim

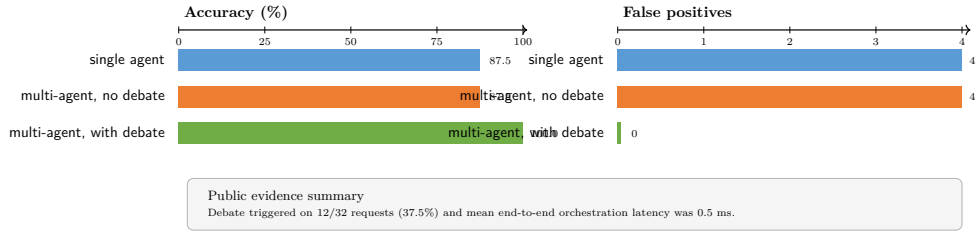


Fig. 4 Summary of the public benchmark and overhead evidence used in this draft.

design: most requests do not require explicit disagreement handling, while a smaller subset benefits from it.

5.5 Limitations of the Current Public Layer

The public evidence should be read with several constraints in mind. First, the corpus size is too small for broad generalization. Second, a benchmark-derived slice is not equivalent to operational traffic. Third, the repository includes a scaffold for a ModSecurity + CRS baseline, but it was not executable in the current environment and is therefore not reported as a competitive result. The comparison is mentioned only to document evaluation intent and to avoid implying that the baseline already ran [19].

5.6 Private Labeled-Alert Reporting Template

The manuscript reserves a second evidence layer for private labeled alerts because that is the correct place to test whether debate reduces false positives on real ambiguity. Table 4 provides the reporting template instead of inventing numbers.

This template keeps the draft honest: the paper is ready for a stronger second evidence layer, but it does not pretend that the layer already exists.

Table 4 Template for the private labeled-alert dataset revision.

Field	Value to report later
Total labeled alerts	TBD
Distinct protected applications	TBD
Label space	benign / suspicious / malicious or mapped binary labels
Time span covered	TBD
Split policy	70/10/20 by time if timestamps exist, else fixed-seed stratified 80/20
Main thesis test	whether debate reduces false positives on ambiguous alerts
Required baselines	single agent, multi-agent without debate, multi-agent with debate
Required error analysis	at least 10 representative false positives before and after debate

6 Discussion and Conclusion

6.1 What the Current Evidence Supports

The public results support a precise and limited conclusion. In the current repository evaluation, debate-driven triage improves request-level precision on ambiguous web threats without materially changing the lightweight orchestration profile of the pipeline. The strongest direct evidence is the removal of four false positives that remain in both non-debate baselines. This aligns with the design intent of the method and with the observable behavior of the code paths that compute disagreement and launch bounded debate.

The system evidence is also meaningful, but should remain conservative. The validated workspace snapshot confirms that the paper is not describing an isolated algorithm detached from a deployable security stack. Authenticated gateways, signed command dispatch, audit-chain retention, and benchmark harnesses are present. However, these artifacts do not justify broad claims about production scale, full LLM latency, or externally validated rollout. The draft therefore frames AetherGuard as a concrete realization environment, not as proof of every historical platform claim.

6.2 Threats to Validity

Three threats are especially important.

First, the public corpus is small and benchmark-derived. It demonstrates mechanism behavior, not broad field performance. Second, the current route-mode latency values describe orchestration in the validated workspace rather than full downstream model latency under all deployment modes. Third, the role-specialization argument is stronger conceptually than empirically in the current draft because the repository does not yet include a separate public ablation that removes role specialization while preserving all other variables.

The planned private labeled-alert layer addresses the first and third limitations most directly. Once that data is available, the paper should add a role-specialization ablation and a targeted error analysis on genuine alert ambiguity.

6.3 Conclusion

This draft reframes AetherGuard into a narrower, more defensible JCST contribution. The paper does not argue that a broad security platform is itself the novelty. Instead, it argues that debate-driven multi-agent judgment is a practical way to improve distributed web threat triage when evidence is ambiguous and operators require an auditable decision trail. The repository already supports this claim with a functioning implementation, a traceable public benchmark slice, and conservative system-overhead evidence. The next revision should strengthen the paper through a private labeled-alert dataset, but even in its present form the draft provides a coherent method-and-system narrative centered on role-specialized debate, auditable outputs, and deployment-aware realization.

Acknowledgements. Acknowledgements are intentionally anonymized in this draft. Replace this paragraph in the non-anonymous revision with funding support, institutional assistance, infrastructure credits, or other brief acknowledgements that are appropriate for the final submission.

Declarations

- **Funding:** To be completed in the non-anonymous revision. The current draft does not attribute any grant support without repository-traceable confirmation.
- **Conflict of interest:** The author declares no competing interests for this anonymized draft.
- **Ethics approval and consent to participate:** Not applicable.
- **Consent for publication:** Not applicable.
- **Data availability:** Public evidence used in this manuscript is traceable to repository artifacts listed in Appendix A. The private labeled-alert layer is reserved for a later revision and is not populated in the current draft.
- **Materials availability:** Not applicable.
- **Code availability:** The implementation surfaces discussed in this paper are present in the repository workspace referenced throughout the manuscript and Appendix A.
- **Author contribution:** To be completed in the non-anonymous revision.

References

[1] D. E. Denning, "An intrusion-detection model," *IEEE Transactions on Software Engineering*, vol. SE-13, no. 2, pp. 222–232, 1987.

[2] M. Roesch, "Snort – lightweight intrusion detection for networks," in *Proceedings of the 13th USENIX Conference on System Administration*, 1999, pp. 229–238.

[3] V. Paxson, "Bro: A system for detecting network intruders in real-time," *Computer Networks*, vol. 31, no. 23–24, pp. 2435–2463, 1999.

[4] M. Doriguzzi-Corin, S. Millar, S. Scott-Hayward, P. Martínez-Julia, and A. Pecorella, "LUCID: A practical, lightweight deep learning solution for DDoS attack detection," *IEEE Transactions on Network and Service Management*, vol. 17, no. 2, pp. 876–889, 2020.

[5] K. Neupane, N. Ene, A. Baird, H. Aksu, and A. S. Uluagac, "Explainable intrusion detection systems (X-IDS): A survey of current methods, challenges, and opportunities," *IEEE Access*, vol. 10, pp. 112436–112469, 2022.

[6] J. Holdbrook *et al.*, "Network-based intrusion detection for industrial and robotics systems: A comprehensive survey," *Sensors*, vol. 24, no. 3, p. 789, 2024.

[7] M. Wang *et al.*, "A survey on large language model based autonomous agents," *Frontiers of Computer Science*, vol. 18, no. 6, p. 186345, 2024.

[8] Q. Wu, G. Bansal, J. Zhang, *et al.*, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," in *Proceedings of the First Conference on Language Modeling*, 2024.

[9] S. Hong, X. Zheng, J. Chen, *et al.*, "MetaGPT: Meta programming for a multi-agent collaborative framework," in *The Twelfth International Conference on Learning Representations*, 2024.

[10] Y. Du, S. Li, A. Torralba, *et al.*, "Improving factuality and reasoning in language models through multiagent debate," in *Proceedings of the Forty-First International Conference on Machine Learning*, 2024, pp. 10889–10908.

[11] J. Huang, X. Chen, S. Mishra, *et al.*, "Large language models cannot self-correct reasoning yet," in *The Twelfth International Conference on Learning Representations*, 2024.

[12] A. Gou *et al.*, "CRITIC: Large language models can self-correct with tool-interactive critiquing," in *The Twelfth International Conference on Learning Representations*, 2024.

[13] Z. Chen *et al.*, "A survey on privacy risks and protection in large language models," *Artificial Intelligence Review*, vol. 58, no. 1, pp. 1–45, 2025.

[14] L. Chen, M. Zaharia, and J. Zou, "FrugalGPT: How to use large language models while reducing cost and improving performance," *arXiv preprint arXiv:2305.05176*, 2023.

[15] I. Ong *et al.*, "RouteLLM: Learning to route LLMs with preference data," *arXiv preprint arXiv:2406.18665*, 2024.

[16] E. Vasilomanolakis, S. Karuppayah, M. Mühlhäuser, and M. Fischer, "On the taxonomy of collaborative intrusion detection," *ACM Computing Surveys*, vol. 47,

no. 4, pp. 1–33, 2015.

[17] M. Barrere *et al.*, "Can machine learning assist human operators in cyber defence? A survey," *ACM Computing Surveys*, vol. 55, no. 7, pp. 1–36, 2022.

[18] OWASP Foundation, "OWASP benchmark project." [Online]. Available: https://owasp.org/www-project-benchmark/. Accessed: Mar. 13, 2026.

[19] OWASP Foundation, "OWASP ModSecurity Core Rule Set." [Online]. Available: https://owasp.org/www-project-modsecurity-core-rule-set/. Accessed: Mar. 13, 2026.

A Artifact Traceability

Table 5 maps the manuscript's numeric and implementation-specific claims to concrete repository artifacts. This appendix is not meant as a publication novelty claim; it is included so that the draft can be reviewed against the current workspace without relying on memory or legacy whitepaper numbers.

Table 5 Traceability map from manuscript claims to repository artifacts.

Manuscript claim	Repository artifact
32 balanced benchmark-derived samples across four web attack families	docs/papers/ieee-access-submission/experimental-results/owasp-web-eval.json
87.5% accuracy and four false positives for `single_agent` and `multi_agent_no_debate`	docs/papers/ieee-access-submission/experimental-results/owasp-web-eval-summary.md
100% accuracy and zero false positives for `multi_agent_with_debate`	docs/papers/ieee-access-submission/experimental-results/owasp-web-eval-summary.md
12/32 debate trigger ratio and 0.75 average debate rounds	docs/papers/ieee-access-submission/experimental-results/owasp-web-eval-summary.md
route-mode orchestration latency values	docs/papers/ieee-access-submission/experimental-results/owasp-web-eval-summary.md
fastpath smoke snapshot of 1329.79 QPS and 21 ms p95 latency	docs/reports/validated-capability-status-2026-03-08.md
disagreement logic using label conflict, Jensen–Shannon divergence, and confidence range	backend/src/ai-agents/core/scoring.ts
bounded debate orchestration and Aristotle arbitration	backend/src/ai-agents/debate/debate-protocol.ts
route-mode names and privacy configuration surface	backend/src/services/platform-config.service.ts
authenticated and rate-limited WebSocket gateway plus audit logging references	backend/src/websocket/agent-gateway.ts